خلاصه کلی مقاله

ما اگر در امبدد سیستم ها به صورت سنتی بخواهیم برنامه نویسی کنیم باید برنامه نویسی سطح پایین انجام دهیم و این کار پیچیدگی کار را زیاد میکند به همین دلیل ترجیح میدهیم از سیستم عامل های بلادرنگ در امبدد سیستم ها استفاده کنیم. این سیستم عامل ها اجرای همزمان کار ها و وظایف را در خود دارند و همچنین با استفاده از کتابخانه ها و API هایی که در اختیارمان میگذارد باعث میشود که در سطح پایین برنامه نویسی نکنیم و از پیچدگی کار و استفاده مجدد از کد تعبیه شده در موارد دیگر بهره مند شویم.

از سیستم عامل های مطرح میتوان به زایفر و Free rtos اشاره کرد که پشتیبانی از نخ ها را نیز دارا میباشند.وقتی صحبت از پشتیبانی از نخ ها میشود یعنی یک فضای حافظه مشترک توسط چندین نخ استفاده میشود که در سیستم های واکنش گرا و تعاملی مدیریت دقیق حافظه دشوار است.به همین خاطر، مدل های جایگزین همزمانی مانند مدل actor یا فرایند های توالی (CSP) – ارتباطی مطرح شده اند که مشکلات همگام سازی و دسترسی به حافظه مشترک را حل میکنند(مشکلاتی مانند data race و deadlock) اما در دو موضوع به مشکل بر میخورند؛ اول آنکه آنها در ارائه قطعیت و زمان بندی به مشکل برمیخورند یعنی این مدل ها نمیتوانند تضمین کنند که رویدادهای سیستم در زمان های دقیق و مشخصی رخ دهند، دوم آنکه نمی توان تضمین کرد که رفتار یک برنامه یا کتابخانه که زمان بندی های خاصی را می طلبد، به شکلی یکسان و بدون تغییر در تمامی پلتفرم ها عمل کند.

در این مقاله استفاده از LF را به عنوان یک راهی برای فراتر رفتن از مدل برنامه نویسی مبتنی بر نخ که توسط RTOS ها رائه شده است را معرفی میکنیم. لینگو فرانکا این امکان را فراهم میکند که تا هم زمانی و زمان بندی را به شیوه ای مستقل از پلتفرم بیان کنیم(منظور این است که برنامه ساخته شده میتواند با کمترین تغییرات بر روی انواع مختلف سیستم های عامل، سخت افزار ها و محیط های نرم افزاری اجرا شود) در این مقاله عملکرد LF در مقایسه با کد C که مبتنی بر نخ میباشد را روی سیستم عامل زیفایر از نظر زمان پاسخگویی (مدت زمانی که طول میکشد تا یک سیستم نسبت به یک رویداد ورودی واکنش نشان دهد) ،دقت زمان بندی(توانایی سیستم در اجرای عملیات ها در زمان های دقیق و از پیش تعیین شده)، گذردهی (تعداد عملیات هایی که یک سیستم میتواند در واحد زمانی تکمیل کند) و اثر پای حافظه(مقدار حافظه ای که برنامه استفاده میکند) مقایسه کنیم.

مثال فرض کنید یک ربات آشپز در یک آشپزخانه تعبیه شده است که باید به طور هم‌زمان کیک بپزد و قهوه درست کند. اگر از RTOS استفاده کنیم، ربات باید بین این دو کار جابجا شود که این می‌تواند باعث تأخیر یا نوسان در زمان‌بندی کارها شود. اما اگر مدل Reactor که توسط LF ارائه شده است را به کار ببریم، ربات می‌تواند به طور دقیق هر فرآیند را با زمان‌بندی مشخص و قابل اتکا انجام دهد و بنابراین همیشه کیک و قهوه را به موقع، بدون تأخیر و با کیفیت مطلوب آماده کند.

بخش اول : معرفی

میکروکنترلرها به طور سنتی با استفاده از زبان سی و برپایه یک مدل همروندی با حافظه مشترک که توسط وقفه ها ارائه میشود برنامه ریزی شده اند. این مدل برنامه ها از قدیم مشهور به دشواری در اجرای صحیح دارند و گاها مشلات و آسیب های جدی ای نیز به وجود آورده است (اگر برنامه به خوبی نوشته نشده باشد، ممکن است در مواقع حیاتی دچار وقفه شود یا مشکلاتی مانند Race condition و deadlock داشته باشند).

سیستم عامل های Real time یک نسخه پیشرفته تر از مدل همروندی با حافظه مشترک را با استفاده از نخ ها ارائه میدهند. با استفاده از نخ ها کاربر میتواند برنامه خود را به چندین فعالیت همزمان تقسیم کند.این سیستم عامل ها یکسری امکانات پایه برای کنترل زمان بندی و همگام سازی میان نخ ها ارائه میدهد. درست مانند وقفه ها که گفته شد، مدل برنامه نویسی مبتنی بر نخ ها نیز عدم قطعیت را به همراه دارد و کار دشوار تطابق دسترسی نخ های مختلف به متغیر های مشترک برنامه را به برنامه نویس واگذار میکند.

یک ابزار همروندی که سیستم عامل های محبوب مثل زفایر به عنوان جایگزین نخ ها ارئه میدهد، کال بک است که از روالهای روتین وقفه فراخوانی میشود. البته حتی این ابزار هم به طور کامل نمیتواند مشکل عدم قطعیت را حل کند(تنها دشواری در مدیریت زمان بندی و همگام سازی دسترسی ها را کاهش میدهد). اما در این روش یک مشکل به نام معکوس سازی جریان کنترل را ایجاد میکند که درک برنامه ها را خیلی مشکل میکند.

در RTOS دقت زمانی ارائه شده برای خیلی از برنامه ها کافی نیست، در واقع RTOS ها از سیستم تیک برای اندازه گیری زمان و زمان بندی استفاده میکند که فاصله زمانی میان آنها قابل تنظیم توسط کاربر است اما معمولا در سطح میلی ثانیه است که خیلی زیاد است! (به عنوان مثال با استفاده از سیستم تیک در دوره های ۱ میلی ثانیه، وضعیت حسگرهای دما چک میشود و در صورتی که دما از حد معینی فراتر برود، نخ های کنترل کننده سیستم های گرمایشی یا سرمایشی فعال میشود).

{منظور از قطعی بودن آن است که هر عملیاتی با ورودی هی مشابه نتیجه ای یکسان داشته باشد. منظور از هم روندی نیز آن است که چندین عملیات یا محاسبات به طور همزمان روی یک سیستم اجرا شوند(یعنی چندین بخش از کد به طور همزمان اجرا شود(با استفاده ازنخ ها، فرایندها یا راکتور ها)).}

دراین مقاله از زبان لینگو فرانکا استفاده میکنیم که امکان همروندی، قطعیت و زمان بندی دقیق را به ما میدهد و مشکلات بالا را حل نموده است. در این زبان شئ هایی به نام راکتور وجود دارد که از طریق پورت ها به همدیگر اطلاعات ارسال میکنند و امکان اجرای همروند در برنامه را فراهم میکنند. همچنین راکتور ها به طور خودکار همروندی را مدیریت میکنند (بدون نیاز به نوشتن کد دستی برای قفل ها یا متغیر های شرطی). همچنین لینگو فرانکا این امکان را میدهد که از زبان هدف خود استفاده نماییم (لینگو فرانکا یک زبان هماهنگ سازی است نه یک زبان برنامه نویسی). در این مقاله قصد داریم تا الگوهای معمول در نرم افزار سیستم های تعبیه شده را با استفاده از مدل همروندی مبتنی بر راکتور زمان بندی شده با نخ ها را مقایسه میکنیم(مانند الگوی جرقه زدایی کلید و ...).

برای مثال، راکتور حسگر ترافیکی ممکن است داده‌های حرکت وسایل نقلیه را دریافت کند و در صورت تشخیص یک حجم ترافیکی زیاد، می‌تواند یک پیام به راکتور چراغ راهنمایی ارسال کند تا زمان سبز بودن چراغ را افزایش دهد. همچنین، اگر تراکم ترافیک به حدی است که باعث احتمال تصادف می‌شود، راکتور حسگر می‌تواند پیامی به راکتور سیستم آلارم ارسال کند تا هشدار لازم را صادر کند.

در ادامه مقاله به بخش های زیر تقسیم بندی شده است :

بخش دوم : راکتور ها در لینگو فرانکا و مطابقت آن برای طراحی سیستم های امبدد را معرفی میکند(الگوهای متداول برنامه نویسی امبدد)

بخش سوم : به بررسی کارهای مرتبط می پردازد.

بخش چهارم : لایه پشتیبانی زفایر را که برای لینگو فرانکا توسعه دادیم معرفی میکند.

بخش پنجم : ما عملکرد لینگو فرانکا روی زفایر را با کد C چند نخی که مستقیما روی زفایر اجرا میشود مقایسه میکنیم.

بخش ششم : نتیجه گیری های بخش پنجم ارائه گردیده است.

هم‌زمانی یا Concurrency تنها مختص سیستم‌های RTOS نیست. این مفهوم در اکثر سیستم‌های عامل مدرن وجود دارد، از جمله در سیستم‌های عملیاتی رومیزی متداول مانند Windows, macOS و لینوکس. در هر یک از این سیستم‌ها، برنامه‌ها می‌توانند به صورت همزمان اجرا شوند.

تفاوت اصلی بین یک RTOS و سیستم‌های عامل عمومی (General-Purpose Operating Systems) به موارد زیر بر می‌گردد:

۱. **پاسخ‌گویی زمانی (Timing Guarantee):**

- **RTOS:** تضمین می‌کند که وظایف و حوادث ضروری در زمان‌های معین و با تأخیر بسیار کم پردازش شوند. RTOSها اولویت‌بندی دقیق و قابل پیش‌بینی از وظایف را فراهم می‌آورند تا اطمینان حاصل شود که وقفه‌های مهم به موقع بررسی و پردازش می‌شوند.

- **سیستم‌های عامل رومیزی:** اگرچه می‌توانند وظایف را به صورت موازی اجرا کنند، اما تعهدات صریح در مورد زمان‌بندی اجرا یا پاسخ به حوادث ارائه نمی‌دهند. در نتیجه، ممکن است در مواقعی که منابع سیستم پر از بار باشند، تأخیر در پاسخ‌گویی رخ دهد.

۲. *عملیات Deterministic:**

- **RTOS:** سیستم‌های RTOS دارای ماهیت Deterministic هستند، به این معنی که رفتار سیستم در طول زمان قابل پیش‌بینی و ثابت است.

- **سیستم‌های عامل عمومی:** این سیستم‌ها ممکن است non-deterministic باشند، زیرا تمرکز آن‌ها روی بهبود کارایی کلی و پشتیبانی از طیف وسیعی از برنامه‌ها و سخت‌افزارها است، نه بر روی رفتار زمانی پیش‌بینی‌پذیر.

۳. **Resource Handling:**

- **RTOS:** مدیریت منابع در RTOSها بسیار سفت و سخت است تا اطمینان حاصل شود که وظایف دارای اولویت بالا به منابع کافی برای اجرای در مواقع مورد نیاز دسترسی دارند.

- **سیستم‌های عامل رومیزی:** مدیریت منابع کمتر تمرکز بر روی اولویت‌های دقیق زمانی دارد و بیشتر در پی بهینه‌سازی استفاده از منابع سیستمی برای تمام فرآیندها و کاربردهای کاربر است.

در حال حاضر، با پیشرفت‌های فناوری و تکنیک‌های پیشرفته زمان‌بندی، برخی از سیستم‌های عامل عمومی قادر به ارائه ویژگی‌های هم‌زمانی و پاسخ‌گویی و حتی قابلیت‌های مشابه RTOSها هستند، ولی همچنان باید توجه داشت که RTOSها برای کاربردهایی که نیازمند قابلیت‌های زمانی بسیار دقیق هستند طراحی شده‌اند.

بخش دوم : برنامه نویسی سیستم های نهفته با LF

در برنامه نویسی مبتنی بر LF همزمانی و همروندی توسط برنامه نویس از طرق ترکیب راکتور ها صورت میگیرد. این راکتورها در واقع هرکدام دارای منطق خاص خود هستند و وقتی یک رویداد خاص رخ میدهد، واکنش خاص از خود نشان میدهند، منطق این واکنش ها در درون خود راکتور محصور شده است. بنابر این در لینگو فرانکا همزمانی میان راکتور های مختلف را کامپایلر و محیط اجرایی LF مدیریت میکند و برنامه نویس نگرانی درمورد موضوعاتی مثل ایجاد قفلها و سمافور ها و ... ندارد.بنابر این راکتورهای مختلف میتوانند به صورت همزمان کارکنند (در صورتی که وابستگی داده ای بینشان وجود نداشته باشد). این موارد باعث میشود که نرم افزار های لینگو فرانکا به صورت پیش فرض قطعی باشد.

زمانی که رویداد ها از نظر منطقی همزمان نیستند، محیط اجرایی لینگو فرانکا تضمین میدهد که آنها باترتیبی مشخص پردازش شوند. این ترتیب که به اصطلاح تگ نام دارد آن رویداد ها را در یک خط زمانی منطقی مرتب قرار میدهد. (مثلا تصور کنید که می‌خواهیم یک برنامه‌ریزی آبیاری باغچه را ایجاد کنیم که هر شب ساعت ۹ به مدت ۱۰ دقیقه روشن می‌شود. اینجا، راکتور ما به یک تایمر وصل است که هر روز ساعت ۹ شب یک رویداد تولید می‌کند. این رویداد "تگ" خاصی دارد که نشان دهنده زمان منطقی آن (ساعت ۹ شب) است. LF می‌تواند به صورت دقیق این زمان را با زمان فیزیکی ساعت سیستم منطبق کند تا آبیاری دقیقا در زمان مشخص شروع شود. همچنین تعریف می‌کنیم که آبیاری باید بعد از ۱۰ دقیقه خاتمه یابد، که این یک ضرب‌الاجل منطقی است. اگر به هر دلیلی آبیاری بیشتر از ۱۰ دقیقه طول بکشد، LF می‌تواند یک واکنش به این تخلف زمانی داشته باشد، مثلا با ارسال یک هشدار به برنامه‌نویس یا خاموش کردن خودکار سیستم آبیاری. این سیستم اطمینان حاصل می‌کند که رویدادها دقیقا مطابق برنامه‌ریزی منطقی و در زمان‌های مشخص اجرا می‌شوند.)

ابزارهای LF نمودارهای تعاملی را در IDE ایجاد میکنند که به درک بهتری از ساختار برنامه منجر میشود. در این نمودار ها راکتورها به صورت جعبه های گرد نشان داده می شوند و... .

اگر بخواهیم لینگو فرانکا را بایک سیستم عامل RTOS مقایسه کنیم میتوان گفت هردو از انتقال پیام و کانتکست سویچ بین اجزا که به صورت همزمان اجرا میشوند پشتیبانی میکند. دو نوع کانتکس سویچ توسط محیط اجرایی لینگو فرانکا انجام میشود، یکی تغییر واکنش، دیگری پیشروی زمان منطقی برنامه است. (مثلا فرض کنید یک ربات خانگی داریم که برای چهار کار مختلف برنامه‌ریزی شده است: جاروب کردن، تحویل محموله، هشدار امنیتی و شارژ خودکار. در هر حالت، ربات نیاز به اطلاعات متفاوتی دارد که از پورت‌های مختلف وارد می‌شوند. به طور مثال، در حالت "جاروب کردن"، اطلاعات از حسگرهای خاک ورودی هستند و در حالت "تحویل محموله"، دستورات از قسمت سرویس تحویل ورودی هستند. LF به این ربات اجازه می‌دهد که با استفاده از جابجایی زمینه و تغییر واکنش سریعاً بین این حالت‌ها جابجا شود، بدون اینکه نیاز باشد داده‌ها کپی شوند؛ زیرا از ارتباطات بدون کپی پشتیبانی می‌کند. همچنین، اگر ربات برای مدتی کار نداشته باشد و به انتظار حوادث بعدی باشد، می‌تواند به حالت خواب کم‌مصرف برود(زمینه پیشروی زمان منطقی) تا زمان فیزیکی به زمان برنامه‌ریزی شده برسد، که این انرژی را صرفه‌جویی می‌کند و بهره‌وری را بهبود می‌بخشد).

۲/۱. الگوهای طراحی سیستم های نهفته :

الگوی متداول طراحی در سیستم های امبدد را میتوان به راحتی با استفاده از زبان LF بیان کرد.

۱. جرقه زدایی دکمه

a. وقتی یک دکمه فشرده میشود، ممکن است چندین ترانزیشن غیر اصلی را ایجاد میکند. ما با استفاده از جرقه زدایی فرایند فیلترکردن ترنزیشن هایی که از نظر زمانی به همدیگر نزدیک هستند را انجام میدهیم.(مثال:بگذارید فرض کنیم در یک سیستم کنترل دسترسی، یک دکمه برای باز کردن درب وجود دارد. هنگامی که کاربر برای ورود به دکمه دست می‌زند، ممکن است به دلیل خصوصیات مکانیکی دکمه و سرعت بالای فشردن و رها کردن، سیگنال‌های اضافی به سیستم ارسال شوند. این امر می‌تواند باعث شود که درب بیشتر از یک بار قفل و باز شود که ناخواسته است و امنیت سیستم را پایین می‌آورد.برای جلوگیری از این مشکل، مهندسان یک روش جرقه‌زدایی را در نرم‌افزار تعبیه می‌کنند. با استفاده از LF، آن‌ها می‌توانند یک "عمل فیزیکی" را تعریف کنند که فشار دکمه را تشخیص دهد و برنامه‌زمانبندی کند که این فشار دکمه در خط زمانی منطقی اجرا شود. با تعیین یک فاصله مینیمم برای این عمل، مثلاً ۱۰ میلی‌ثانیه، محیط اجرایی LF تمام فشارهایی که این فاصله را نقض می‌کنند را رد می‌کند و فقط یک فشار معتبر را برای باز کردن درب قبول می‌کند. با این روش، فقط یک سیگنال باز شدن به سیستم کنترل درب ارسال می‌شود حتی اگر کاربر بارها و بسیار سریع به دکمه فشرده باشد.)

b. وقتی که یک دکمه فشار داده می‌شود، ممکن است چندین تغییر وضعیتِ ناخواسته‌ی باز و بسته ایجاد کند. «اِرتعاش‌زدایی» (Debouncing) فرآیندی است برای فیلتر کردن این تغییرات وضعیت که خیلی به هم نزدیک هستند و احتمالا به خاطر یک فشار دادن یا رها کردن واقعی دکمه نیستند. زمان‌بندی مبتنی بر زمان (LF's timed semantics) LF اینگونه الگوها را به راحتی بیان می‌کند. برای ثبت رویدادهای غیر همزمان، مثل فشار دادن دکمه‌ها، و قرار دادن آن‌ها در خط زمانی منطقی LF، از عمل فیزیکی استفاده می‌شود، که می‌تواند با یک نخ خارجی یا ISR برنامه‌ریزی شود.برچسب چنین رویدادی براساس زمان فیزیکی سیستم استوار است، نه براساس زمان منطقی فعلی ران‌تایم (مانند یک رویداد که بر روی یک عمل منطقی برنامه‌ریزی شده باشد)، پس یک عمل فیزیکی می‌تواند فاصله‌ی زمانی حداقلی را اختصاص دهد که باعث می‌شود زمان‌بند برنامه‌هایی که خیلی به هم نزدیک برنامه‌ریزی شده‌اند را رد کند. شکل ۱ نشان می‌دهد که چگونه این امر در هدف C از زبان برنامه‌نویسی Lingua Franca بیان می‌شود. عمل فیزیکی «فشار دکمه» (button_press) فاصله زمانی حداقلی ۱۰ میلی‌ثانیه دارد و سیستم ران‌تایم عمل ارتعاش‌زدایی را مدیریت می‌کند.

۲. تجزیه سنسور

a. تصور کنید یک سنسور دما وجود دارد که اطلاعات خود را از طریق یک پروتکل ارتباطی سریال به یک میکروکنترلر ارسال می‌کند. این بسته‌های اطلاعاتی شامل عناصر مختلفی هستند، مانند شناسه سنسور، مقدار دما، و یک کد تأیید برای چک کردن صحت داده‌ها.

ری‌اکتور LF به کار رفته در میکروکنترلر ابتدا منتظر دریافت بایت‌های همگام‌سازی می‌ماند که ابتدای یک بسته جدید را نشان می‌دهند. به محض دریافت این بایت‌های خاص، ری‌اکتور حالت خود را تغییر می‌دهد تا طول بسته را بخواند (ReadLength). پس از خواندن و محاسبه طول، ری‌اکتور حالت خود را دوباره تغییر می‌دهد به ReadPacket، کل بسته را می‌خواند، داده‌ها را تجزیه می‌کند و بسته تجزیه‌شده را به بقیه سیستم ارسال می‌کند. این نحوه کار سیستم می‌تواند به حصول اطمینان از درستی و کارایی پردازش داده‌های دریافتی از سنسورها کمک کند.

b. متن به برنامه‌ای در زبان Lingua Franca (LF) اشاره دارد که برای تجزیه و تحلیل داده‌های دریافتی از یک حسگر خارجی که از طریق یک رابط ارتباط دیجیتال متصل شده است، مورد استفاده قرار می‌گیرد. در این سیستم، داده‌های دریافتی به صورت جریانی از بایت‌ها هستند و تجزیه آنها به‌طور مناسب به شکل یک ماشین حالت بیان شده است.لینگو فرانکا قابلیت‌های بومی برای کدنویسی ماشین‌های حالت را از طریق چیزی به نام رآکتورهای مدال دارد. رآکتور SensorParsing دارای یک پورت ورودی برای بایت‌های دریافتی و یک پورت خروجی برای بسته اطلاعاتی تجزیه‌شده است. چگونگی واکنش رآکتور به داده‌های ورودی بستگی به حالت فعلی‌اش دارد. در حالت اولیه، موسوم به ReadSync، بایت‌های ورودی با شناسه سنکرون‌سازی (sync ID) شناخته شده‌ی بسته مقایسه می‌شوند. به هنگام یافتن بایت‌های سنکرون‌سازی، رآکتور تغییر حالت به ReadLength را انجام داده و در اینجا اندازه بسته را می‌خواند. زمانی که اندازه رمزگشایی شد، رآکتور به حالت ReadPacket می‌رود، بسته را می‌خواند و نتیجه را در پورت خروجی pkt تولید می‌کند.

۳. حلقه های کنترلی

a. یکی از الگوهای طراحی رایج در حوزه‌هایی مانند رباتیک و کنترل، حلقه‌های کنترل دقیق است. در نظر بگیرید برنامه LF در تصویر ۳ نشان داده شده است. این برنامه شامل یک واکنش‌گر سنسور است که هر ۱ میلی‌ثانیه از یک سنسور مقدار خوانی می‌کند. مقادیر سنسور به واکنش‌گر پردازش (Processing) منتقل می‌شود که یک الگوریتم کنترلی را پیاده‌سازی می‌کند. سرانجام، نقاط تنظیم کنترل محاسبه شده توسط Processing بر روی عملگرها در واکنش‌گر Actuator اعمال می‌شوند. توجه کنید که دو ضرب‌الاجل وجود دارد. اول، واکنش سنسور با ضرب‌الاجل ۲۰ میکروثانیه مرتبط است. این نحوه‌ای برای بیان حد مجاز پرش‌های زمانی (jitter) که در خواندن‌های سنسورمان می‌پذیریم، است. به همین ترتیب، واکنش‌گر Actuatorنیز ضرب‌الاجلی به مدت ۱ میلی‌ثانیه دارد. این مدت با دوره حلقه کنترل مطابقت دارد و بیان می‌کند که بار اضافه‌ای که از زمان دریافت خوانش از سنسور شروع شده و تا کمی قبل از فراخوانی Actuator ادامه دارد، باید کمتر از ۱ میلی‌ثانیه باشد. معناشناسی زمانی LF یک راه شهودی برای بیان چنین حلقه‌های کنترلی و نیازمندی‌های زمان‌بندی آن‌ها فراهم می‌کند.

b. **مثال**:تصور کنید شما سیستمی را برای کنترل یک بازوی رباتیک طراحی کرده‌اید که باید با دقت بالا حرکت کند. شما یک واکنش‌گر "سنسور" دارید که موقعیت فعلی بازو را می‌خواند، یک واکنش‌گر "پردازش" که محاسبات کنترلی را برای تعیین حرکت بعدی بازو

انجام می‌دهد و در نهایت یک واکنش‌گر "عملگر" که این دستورات را بر روی موتورهای رباتیک اعمال می‌کند. برنامه به گونه‌ای تنظیم شده است که سنسور باید در مدت زمان کمتر از ۲۰ میکروثانیه مقدار خوانی را انجام دهد تا حداکثر دقت را حفظ کند. همچنین، عملگر باید هر ۱ میلی‌ثانیه یک بار از دستورات محاسبه شده توسط پردازش استفاده کند تا حرکت بازو به طور یکنواخت و با کنترل دقیق انجام شود.این ضرب‌الاجل‌ها و نیازمندی‌های زمانی را می‌توان به وضوح و به شکل استانداردی با استفاده از LF بیان کرد.

بخش سوم : کارهای مرتبط؟ کجای مقاله استفاده میشود؟

در عرصه تحقیقاتی، کارهای بسیاری صورت گرفته است که به دنبال ارتقای سطح انتزاع برای طراحی نرم‌افزارهای تعبیه‌شده هستند.SynchronVM" " یک ماشین مجازی است که روی سیستم عامل‌های Zephyr یا ChibiOS اجرا می‌شود و مختص میکروکنترلرها طراحی شده است.

مفهوم SynchronVM در دنیای امبدد سیستم ها به ایجاد یک لایه انتزاعی سطح بالاتر برای برنامه ریزی فرایند ها و مدیریت کارها باتمرکز بر همزمانی و ارتباطات میپردازد. با استفاده از این ماشین مجازی، توسعه دهندگام میتوانند یک پنجره زمانی تعریف کنند که در آن انتقال پیام ها به صورت همزمان اتفاق میفتد. یعنی میتوانیم مثلا بگوییم "این داده باید ظرف مدت ۱۰۰ میلی ثانیه پس از جمع آوری به فرایند بعدی ارسال شود" این پنجره زمانی به سیستم اجازه میدهد تا زمان اجرای فرایند ها را برنامه ریزی کند، اما فاقد تعریف دقیق و مشخصی از رفتار زمانی است که باید رعایت شود.

میکرو پایتون یک پروژه است که زبان برنامه نویسی سطح بالا، مفسری و دارای مدیریت حافظه خودکار مانند پایتون را به میکروکنترلر ها میاورد. مفسر این پروژه از REPL پشیتبانی میکند که ساخت نمونه های اولیه پروتوتایپ را ساده و سریع میکند. میکروپایتون به حافظه فلش با حجم حدود ۲۵٦ کیلوبایت نیاز دارد و به دلیل گاربیج کالکتور و داینامیک تایپینگ، برای برنامه های ریل تایم و برنامه های safety-critical مناسب نمیباشد!

Medusa هم یک زبان برنامه نویسی مبتنی بر مدل اکتور میباشد.

زبان برنامه نویسی XC برپایه زبان سی طراحی شده، برای معماری سی پیو XCore از شرکت Xmos ایجاد شده است. Xc مثل SynchronVM بر مدل CSP استوار است (CSP یک مدل برنامه نویسی موازی و همزمان است، در این مدل، فرایند های میتوانند به شکل موازی اجرا شوند).

قسمت چهارم : طراحی و اجرا

ما برای استفاده از زبان برنامه نویسی لینگو فرانکا از سیستم عامل زفایر استفاده میکنیم زیرا بورد های زیادی را پشتیبانی میکند، درایور های مفید زیادی دارد مانند بلوتوث، اترنت، باس و ... .

در این مقاله تمرکز روی اجرای سینگل ترد لینگو فرانکا در زبان C میباشد. در ادامه مواردی که باید روی زفایر توسط لینگو فرانکا پیاده سازی شود توضیح داده میشود.

قسمت پنجم : ارزیابی

ارزیابی با استفاده از کیت توسعه NPRF۵۲ از شرکت نوردیک انجام شد. این کیت دارای یک میکروکنترلر arm با ٦٤ کیلوبایت رم و ۵۱۲ کیلو بایت حافظه فلش است که با فرکانس ٦٤ مگاهرتز کار میکند. بنچمارک با استفاده از ابزار ساخت West نسخه ١٤ و با استفاده از نسخه ۳٫۲ زفیر کامپایل شدند. درادامه ارزیابی ها در چند بخش مقایسه میشود.

۱. ردپای حافظه : ردپای حافظه یک ویژگی حیاتی برای بیشتر برنامه های میکروکنترلری است. در ادامه، ما ارزیابی دقیقی از هردو ردپای حافظه فلش و رم LF را ارائه میدهیم.

۱. سایز کد : ردیف بالایی جدول ۱ اندازه کد اجزای مختلف LF را نشان میدهد. به صورت کلی جمع فضای مورد نیاز در جدول مشخص است.
۲. سایز داده ها : ردیف پایین جدول ۲.؟؟

۲. دقت زمان بندی : پیاده سازی LF دارای دقت زمانی خوب است اگر زمانی فیزیکی که یک واکنش در آن اجرا میشود، نزدیک به زمان منطقی باشد که برای اجرای آن برنامه ریزی شده است.

۱. مولد موج مربعی : برای آزمایش دقت زمانی، برای تولید یک موج مربعی با فرکانس یک کیلو هرتز روی یک پین خروجی با استفاده از تایمرهای سخت افزاری پیکربندی شد.
۲. ؟
۳. ؟

۳. بهره وری : برای ارزیابی عملکرد عمومی LF روی میکروکنترلرها، ما یک زیر مجموعه مرتبط از بنچمارک‌های Thread-Metric و Savina را در مقایسه با یک پیاده‌سازی استاندارد که از APIهای نخ (thread) زفیر استفاده می‌کند اجرا کردیم.

۱. بنچمارک نخ :
۲. ساوینا : مجموعه بنچمارک های ساوینا یک مجموعه بنچمارک مبتنی بر اکتور است. این بنچ مارک ها بیشتر بر روی برنامه هایی با شدت محاسباتی تمرکز دارند تا برنامه هایی که بر IO سنگین هستند.

قسمت ششم : نتیجه گیری